

Keywords: token cues • base models • reasoning • causal data intervention

Base Models Can Reason By Taking a Cue From Training Data

A Single Starting Token Can Raise Math Accuracy by 36 pp in a Base Model—and the Effect Is Causally Traceable to Training Data, Not Model Architecture

By Sophie L. Wang, Amil Dravid, Rulin Shao, Kevin Farhat, Sewon Min & Alexei A. Efros (UC Berkeley)

arXiv:2610.06851 • October 2026

Base language models harbor latent reasoning competence that is largely dormant until activated by a specific starting token—a *reasoning cue* whose effect is not semantic but distributional, mechanically traceable to the cluster of step-by-step reasoning documents in the training corpus that bear the same token.

AT A GLANCE

Problem: Reinforcement learning post-training is costly; the mechanism by which it elicits reasoning chains is poorly understood.

Why it matters: If RL merely selects regions of a base model’s training distribution, cheaper interventions should reproduce the gains.

Method: Systematic sweep over starting-token cues; causal data interventions to add or remove reasoning associations.

Result: The cue “.\n\nOkay” raises Olmo-3-7B MATH-500 from 42% to 78%; causal interventions confirm the training-data mechanism.

Evidence: Evaluated on MATH-500, coding benchmarks, and multiple frontier base models; causal interventions with data ablations.

The Big Picture

Large language models trained with reinforcement learning from verifiable rewards (RLVR) achieve dramatic performance gains on mathematical reasoning and code generation. Yet the precise mechanism has remained unclear: does RL teach the model new reasoning skills, or does it teach the model *when* to deploy skills that the base model already possesses?

This paper proposes a third, more concrete answer: RL teaches the model to begin its response with particular starting tokens—reasoning cues—that route generation

toward the region of the training distribution in which careful step-by-step reasoning documents reside. The paper provides both correlational and causal evidence for this mechanism.

Why Existing Approaches Fall Short

RL post-training is expensive. RLVR requires generating many rollouts and computing rewards, multiplying training compute relative to supervised fine-tuning. If the mechanism is a distributional routing effect, a cheaper intervention should suffice.

Prompt engineering is empirical. Prior chain-of-thought work shows that phrases like “Let’s think step by step” improve performance, but attributes the effect to semantic instruction following—a hypothesis that predicts the effect should generalize to semantically equivalent phrases, which the paper tests and finds false.

Base model latent capacity is underutilized. The default starting token chosen by a practitioner is arbitrary; this work shows it has outsized influence on the quality of the continuation.

Core Mechanism

A **reasoning cue** is a short prefix c placed at the start of a base model’s response. The central finding is that effective cues are not semantically special: their power comes from co-occurring in the training corpus with step-by-step reasoning chains, not from the meaning of the words themselves.

When a cue c appears at the start of the model’s response, next-token prediction conditions on c and selects continuations from the region of the training data distribution where c was associated with careful, multi-step reasoning. This makes the cue a *distributional selector* rather than a semantic instruction.

Technical Formulation

Let $\mathcal{M}$ be a base LLM with vocabulary $\mathcal{V}$. For a problem q and cue string $c \in \mathcal{V}^*$:

$$\text{Acc}(c) = \mathbb{E}_q[\text{Correct}(\mathcal{M}(q, c))]$$

The sweep identifies the optimal cue $c^* = \arg \max_c \text{Acc}(c)$. For the causal intervention, let $D_c \subseteq \mathcal{D}_{\text{train}}$ be training documents whose response begins with c . The paper shows:

$$\text{Acc}_{D \setminus D_c}(c) \ll \text{Acc}_D(c)$$

Conversely, augmenting the training data with arbitrary-token-prefixed reasoning traces $\tilde{D}_c$ creates a new effective cue:

$$\text{Acc}_{D \cup \tilde{D}_c}(c') \approx \text{Acc}_D(c^*)$$

Learning or Inference Procedure

Finding effective cues:

1. Enumerate a large set of candidate starting tokens and phrases.
2. For each cue, generate responses on a held-out problem set.
3. Score responses and rank cues by mean accuracy.

Causal confirmation:

1. Identify training documents co-occurring with the effective cue.
2. Remove those documents and retrain (or fine-tune) a model.
3. Verify that the cue’s effect disappears with the associated data.

Creating custom cues:

1. Collect reasoning traces; prepend an arbitrary token to each.
2. Fine-tune on these modified traces.
3. The arbitrary token becomes an effective cue.

What the Guarantee Says

The guarantee is empirical: across all tested base models and cue candidates, the reasoning benefit of any cue is causally traceable to its associated training documents. Removing those documents degrades the cue; adding documents that associate an arbitrary token with reasoning creates a new effective cue. The mechanism is not model-specific—it holds for Olmo-3-7B, Qwen3-14B, and several other frontier base models.

Experimental Findings

- **Olmo-3-7B:** the cue “.\n\nOkay” raises MATH-500 pass@1 from **42% to 78%**, matching the RL-trained counterpart.
- **Qwen3-14B:** “Alright,” raises MATH-500 from **72% to 87%**, also competitive with RL.
- **Code generation:** analogous cues improve pass@ k metrics on coding benchmarks.
- **Causal intervention:** removing documents associated with the effective cue eliminates its accuracy benefit.
- **Custom cue creation:** the word “chicken,” after fine-tuning on reasoning traces prefixed with it, becomes an effective cue.

Ablations and Interpretation

Semantic vs. distributional hypothesis: Replacing the effective cue with semantically similar phrases that

do not appear in the relevant training data produces no reasoning gain, confirming the distributional mechanism.

Context length stability: The cue effect holds even with few-shot examples in context, ruling out attention-dilution as an alternative explanation.

Multiple cues: Combining multiple effective cues does not additively compound gains, indicating they tap overlapping training data regions.

Implications for RL: Since base-model cue performance matches RLVR, the paper argues that RL primarily teaches models to start responses in the step-by-step reasoning region of their training distribution—not to acquire fundamentally new reasoning capabilities.

Reference

Sophie L. Wang et al. **Base Models Can Reason By Taking a Cue From Training Data.** arXiv:2610.06851, October 2026.

<https://arxiv.org/abs/2610.06851>

Keywords: web agents • conformal prediction • reinforcement learning • test-time scaling

CLIFT: Conformal Self-Verification for Web Agent Training and Test-Time Scaling

A Conformally Certified URL-Aware Verifier Replaces Costly Frontier-LM Supervision with Dense Per-Step Signals and Enables Reliable Test-Time Scaling Without External Oracles

By Yifan Zhang, Yutong Dai, Viraj Prabhu, Zhiyuan Hu, Ran Xu & Zeyuan Chen (Georgia Institute of Technology; University of Texas at Austin)

arXiv:2610.06829 • October 2026

Web agent training faces a fundamental credit-assignment problem: binary task-success rewards are sparse, while frontier-LM judges providing denser step-level feedback are expensive and unavailable at deployment. CLIFT solves both at once with a conformally calibrated URL-aware verifier that supplies non-negative per-step evidence during training and drives reliable test-time scaling without querying any external oracle.

AT A GLANCE

Problem: Sparse binary task rewards leave most intermediate actions unscored; frontier-LM judges are too expensive for every training step.

Why it matters: Dense, calibrated per-step signals accelerate web agent training and enable reliable test-time compute scaling.

Method: Natural-language verification questions grounded in browser state, calibrated by conformal prediction; same verifier reused at inference for Conformal Test-time Scaling.

Result: Higher task success on WebArena and VisualWebArena; 8–12 pp gain from test-time scaling with $N=8$ trajectories, no oracle needed.

Evidence: WebArena and VisualWebArena benchmarks; ablations on calibration, question quality, and verifier model size.

The Big Picture

Reinforcement learning from task-completion rewards is the dominant paradigm for training web agents, but it suffers a fundamental supervision bottleneck: a twenty-step trajectory that ultimately succeeds returns the same sparse positive signal regardless of which intermediate ac-

tions were actually useful. Frontier language model judges can provide step-level feedback, but querying GPT-4o at every step during training is cost-prohibitive and creates deployment-time dependencies on proprietary APIs.

CLIFT (Conformal Learning for Instruction Following at Test time) addresses both problems simultaneously. During training, it replaces frontier-LM supervision with a small verifier whose outputs are calibrated by conformal prediction, guaranteeing non-negative evidence in expectation. At inference, the same frozen verifier drives Conformal Test-time Scaling (CTS), selecting among candidate trajectories without any external oracle.

Why Existing Approaches Fall Short

Binary rewards are sparse. An agent that completes a task in twenty steps receives no information about which individual steps contributed. This makes gradient estimation difficult and exploration slow.

LM judges are expensive and API-dependent. Querying a frontier model at every step dramatically increases training cost and creates an availability dependency that is absent at deployment.

Heuristic verifiers lack calibration. Rule-based verifiers checking URL patterns or DOM state can fire incorrectly; without calibration their signals may actively mislead training.

Test-time scaling is uncertified. Best-of- N and chain-of-thought approaches for agents offer no statistical guarantee that more compute improves reliability.

Core Mechanism

CLIFT introduces two complementary components:

Conformal Self-Verification (training): Natural-language verification questions (e.g., “Is the agent on the correct URL?”, “Has the required form field been filled?”) are generated from task descriptions and grounded in the current browser state (URL and DOM excerpt). A small verifier answers these questions and assigns per-step scores. Conformal prediction calibrates the verifier’s confidence so it fires only when it can certify its answer at a pre-specified error rate—yielding a conformally certified, URL-aware verifier with a formal coverage guarantee.

Conformal Test-time Scaling (CTS, inference): The same frozen verifier bank scores all candidate trajectories sampled by the agent. CTS selects the trajectory with the highest cumulative conformal evidence, enabling reliable test-time scaling with no frontier-model dependency.

Technical Formulation

Let $s_t = (u_t, d_t)$ be the browser state at step t (URL and DOM excerpt), a_t the agent’s action, and $Q = \{q_1, \dots, q_K\}$ a bank of verification questions derived from the task description.

The verifier model v_ϕ scores each question-state pair:

$$p_{k,t} = v_\phi(q_k \mid s_t, a_t) \in [0, 1]$$

Conformal calibration with coverage $1 - \alpha$ produces a threshold τ such that:

$$\Pr[\text{verifier incorrect}] \leq \alpha$$

The per-step evidence signal used during training is:

$$r_t = \sum_{k=1}^K \mathbf{1}[p_{k,t} \geq \tau] \cdot w_k$$

where w_k are task-specific weights. This signal is always non-negative (evidence of progress, never a penalty), so it supplements rather than replaces the sparse task-completion reward.

For CTS at inference, given N sampled trajectories $\{\tau^{(i)}\}_{i=1}^N$, the selected trajectory is:

$$\hat{\tau} = \arg \max_i \sum_t r_t^{(i)}$$

Learning or Inference Procedure

Training:

1. Generate a bank of verification questions per task type.
2. Calibrate the verifier on a held-out set to determine τ at desired coverage $1 - \alpha$.
3. Run RL with task-completion reward augmented by conformal per-step evidence r_t .

Test-time scaling:

1. Sample N trajectories from the policy.
2. Score each step with the frozen conformal verifier.
3. Return the trajectory with the highest cumulative evidence.

What the Guarantee Says

By construction of conformal prediction, the verifier provides a marginal coverage guarantee: the probability that the verifier incorrectly certifies an incorrect action as correct is at most α (typically $\alpha = 0.1$). This ensures per-step signals are non-negative in expectation, preserving policy improvement under the RL update. The CTS selection guarantee is weaker, but conformal calibration ensures the selected trajectory is at least as likely to be correct as random selection would predict.

Experimental Findings

- **WebArena and VisualWebArena:** CLIFT-augmented RL achieves significantly higher task success than RL with binary reward alone.
- **Evidence density:** The conformal verifier fires at roughly 40–60% of steps, providing substantially denser credit assignment than binary rewards.
- **CTS efficiency:** $N=8$ trajectories with CTS selection improves success rate by **8–12 percentage points** over single-trajectory evaluation, with no frontier-model query.
- **Calibration quality:** The conformally calibrated verifier achieves coverage close to the nominal $1 - \alpha$ across all task types.

Ablations and Interpretation

Calibration vs. uncalibrated verifier: Using the raw verifier without conformal calibration produces noisy per-step signals that can harm training; calibration is critical to the method’s benefit.

Question quality: Generic verification questions degrade performance; the quality of the task-specific question bank governs verifier coverage.

Number of CTS samples: Performance improves with more trajectories up to $N=16$; beyond that, gains are marginal, suggesting the verifier reaches its discrimination ceiling.

Verifier model size: A 1B-parameter verifier delivers comparable quality to a 7B verifier for structured URL-aware verification, indicating that small specialized models are sufficient.

Reference

Yifan Zhang, Yutong Dai, Viraj Prabhu, Zhiyuan Hu, Ran Xu & Zeyuan Chen. **CLIFT: Conformal Self-Verification for Web Agent Training and Test-Time Scaling**. arXiv:2610.06829, October 2026.
<https://arxiv.org/abs/2610.06829>

Keywords: world models • hierarchical planning • JEPA • latent representation

H-JEPA: End-to-End Learning of Hierarchical World Models for Visual Planning

Stacking Action-Conditioned JEPAs Hierarchically Raises Visual AntMaze Success from 18% to 73% by Letting Each Level Predict Farther Ahead in Its Own Latent Space

By Wancong Zhang, Basile Terver, Michael Rabbat, Yann LeCun & Randall Balestriero (Meta AI; New York University)

arXiv:2610.06805 • October 2026

Flat world models trained at a single temporal resolution conflate the timescales of muscle-level control, motion planning, and goal-directed navigation, forcing planners to reason about irrelevant visual detail. H-JEPA stacks action-conditioned JEPAs so that higher levels automatically discard fast-varying sensory noise and retain the slow, task-relevant state that long-horizon planning depends on—without any manual specification of sub-goals or timescale boundaries.

AT A GLANCE

Problem: Single-level world models force planners to reason at one temporal resolution, wasting capacity on irrelevant low-level detail during long-horizon planning.

Why it matters: Robotics, navigation, and embodied AI require simultaneous reasoning across sub-second control and minute-scale goals.

Method: Stack L action-conditioned JEPAs; each level predicts k steps ahead in the latent of the level below; train all levels jointly end-to-end with a VICReg-style regularizer.

Result: A 3-level hierarchy raises Visual AntMaze success from **18% to 73%** while reducing planner compute.

Evidence: Visual AntMaze, robotic manipulation, and real-robot planning on the DROID dataset.

The Big Picture

Long-horizon planning requires simultaneous reasoning across radically different timescales: sub-second muscle control, second-scale motion planning, and minute-scale goal navigation. A flat world model trained at a single temporal resolution cannot separate these concerns—it must predict every future state at the same resolution, forcing the planner to process irrelevant low-level visual detail even when making high-level route decisions.

H-JEPA (Hierarchical Joint Embedding Predictive Architecture) resolves this by stacking multiple levels of action-conditioned JEPAs, each operating at a coarser timescale than the one below. The hierarchy is not hand-designed: timescale separation emerges automatically from end-to-end training when the world contains factors that evolve at different speeds.

Why Existing Approaches Fall Short

Flat world models are computationally wasteful.

A single-level model predicts every future state at native resolution, burdening the planner with fine-grained sensory detail even during high-level decision making.

Hand-designed hierarchies require manual sub-goals.

Options frameworks and HIRO-style methods pre-specify the timescale structure and require human-designed sub-goal spaces, sacrificing end-to-end trainability.

Generative hierarchical models are fragile.

Pixel-prediction hierarchies accumulate errors across levels and are sensitive to the choice of reconstruction loss.

Flat JEPA models remain at a single timescale.

V-JEPA and related work learn expressive single-level latent predictors; extending them to multiple timescales in an end-to-end trainable manner is non-trivial.

Core Mechanism

H-JEPA stacks L levels of action-conditioned JEPAs:

- **Level 1** operates at the native frame rate, predicting the latent of the next frame from the current latent and action.
- **Level $\ell > 1$** operates at a k -times-coarser timescale. It receives the temporally pooled latent from level $\ell-1$ and predicts that latent k frames ahead.
- **End-to-end training** jointly minimizes a JEPA objective at all levels: minimize distance between predicted latent and the stop-gradient target from the level below, regularized by a VICReg-style term to prevent representational collapse.

The key emergent property is *timescale separation*: higher levels learn to discard fast, unpredictable visual detail and retain slow, task-relevant state (object positions, goal proximity) without explicit supervision.

Technical Formulation

Let z_t^ℓ denote the latent at level ℓ and time t . Temporal pooling P_k averages k consecutive latents. The H-JEPA target at level ℓ is:

$$z_t^\ell = P_k \left(z_{t-k}^{\ell-1}, \dots, z_{(t+1) \cdot k - 1}^{\ell-1} \right)$$

The predictor f_ϕ^ℓ at level ℓ predicts k steps ahead:

$$\hat{z}_{t+1}^\ell = f_\phi^\ell(z_t^\ell, a_{t:k:(t+1) \cdot k})$$

The per-level JEPA loss is:

$$\mathcal{L}^\ell = \|\hat{z}_{t+1}^\ell - \text{sg}(z_{t+1}^\ell)\|^2 + \lambda \cdot \mathcal{L}_{\text{VICReg}}(z^\ell)$$

The total objective sums over all levels:

$$\mathcal{L}_{\text{H-JEPA}} = \sum_{\ell=1}^L \mathcal{L}^\ell$$

Planning applies a latent MPPI or MCTS planner at the highest level L , scoring rollouts in the level- L latent space—dramatically reducing compute per planning node relative to planning at level 1.

Learning or Inference Procedure

Training:

1. Collect trajectories $(o_1, a_1, \dots, o_T, a_T)$.
2. Encode observations to $z^0 = g_\psi(o)$; propagate up: $z^\ell = P_k(z^{\ell-1})$.
3. Train all levels jointly with $\mathcal{L}_{\text{H-JEPA}}$.
4. Optionally add inverse-dynamics heads at each level.

Planning at test time:

1. Encode current observation; propagate to level L .
2. Run MPPI or MCTS in the level- L latent space.
3. Execute the first action; receive next observation; repeat.

What the Guarantee Says

No formal convergence guarantee is provided. The central empirical claim is that timescale separation emerges automatically from end-to-end training when the data contains factors that evolve at different speeds. This is validated by probing each level’s representations for factors of variation at different timescales. Ablations confirm that a flat JEPA with the same total parameter count does not match the hierarchical planner.

Experimental Findings

- **Visual AntMaze:** A 3-level hierarchy raises success rate from **18% (flat JEPA) to 73%** while requiring less planner compute.
- **Manipulation:** H-JEPA outperforms flat JEPA on robotic pick-and-place and peg-insertion tasks requiring multi-step coordination.
- **DROID (real robot):** With inverse-dynamics supervision, H-JEPA improves offline planning fidelity on diverse real-robot demonstration data.

- **Timescale analysis:** Level-3 representations encode task-relevant state (goal proximity, object position) almost entirely; level-1 is dominated by low-level visual texture.

Ablations and Interpretation

Number of levels: Performance improves consistently from 1 to 3 levels; 4 levels adds marginal gain but increases training complexity.

Pooling stride k : Smaller strides ($k=2$) help tasks with fast-changing task-relevant state; larger strides ($k=8$) help tasks with very long planning horizons.

Timescale separation: Probing confirms the hierarchy: level 1 is dominated by visual texture; level 3 is nearly entirely task-state driven.

Comparison to hand-designed hierarchies: H-JEPA matches or exceeds options-based and HIRO-style methods on standard benchmarks while requiring no manual specification of sub-goals or timescale boundaries.

Reference

Wancong Zhang, Basile Terver, Michael Rabbat, Yann LeCun & Randall Balestriero. **H-JEPA: End-to-End Learning of Hierarchical World Models for Visual Planning.** arXiv:2610.06805, October 2026.
<https://arxiv.org/abs/2610.06805>

Keywords: data selection • post-training • gradient alignment • LLM fine-tuning

LESSER: Post-Training Data Selection with Output-Layer Gradients

Output-Layer Gradients Are a 9.7× Cheaper Proxy for Full-Gradient Data Selection While Preserving Batch-Level Alignment and Downstream Performance

By Lyuxin David Zhang, Eric Wong, Surbhi Goel & Anton Xue (University of Pennsylvania)

arXiv:2610.03702 • October 2026

Gradient-based data selection for LLM post-training requires an expensive full-backward pass for every candidate sample; LESSER shows that restricting gradient computation to the output layer reduces cost by up to 9.7× while matching full-gradient performance, because batches selected by both methods remain well-aligned with the validation gradient even when individual sample rankings differ.

AT A GLANCE

Problem: Full-parameter gradient computation for post-training data selection is intractable at scale.

Why it matters: The right training data can substantially improve downstream performance; efficient selection unlocks practical use.

Method: Restrict gradient computation to the output (language-model head) layer only; use as a drop-in wrapper for any selection algorithm.

Result: 9.7× FLOP reduction for SFT and 3.0× for RL selection, with no downstream performance loss.

Evidence: Evaluated across multiple gradient-based selection methods on SFT and RL post-training benchmarks.

The Big Picture

The choice of which data to use for post-training an LLM substantially shapes its downstream capabilities. Gradient-based data selection formalizes this choice: rank candidate data points by how well their per-sample gradients align with the gradient on a small, high-quality validation set, then train on the top-ranked examples. The method is principled and effective but computationally prohibitive at scale—each candidate requires a full backward pass through a billion-parameter model.

LESSER (Last-layer Efficient Sample Selection via

Output-layer gRadients) asks: does the output layer’s gradient contain enough information to perform effective selection, at a fraction of the cost of backpropagating through the entire network?

Why Existing Approaches Fall Short

Full-gradient cost is prohibitive. For a model with N parameters and a candidate pool of M samples, gradient-based selection requires $O(NM)$ work. At $N = 7 \times 10^9$ and $M = 10^5$, this is impractical without approximation.

Embedding-layer proxies introduce extra hyperparameters. LESS and related methods use LoRA projections of embedding-layer gradients, introducing a rank hyperparameter and a random-projection step that adds engineering overhead.

Sample-level alignment is not the operative quantity. Prior work implicitly assumes that a proxy must rank individual samples identically to full gradients. LESSER demonstrates that batch-level alignment—the quantity that governs gradient descent quality—is preserved even when sample rankings differ.

Core Mechanism

LESSER restricts gradient computation to the **output layer** (language model head)—the final linear projection W_{head} from hidden states to vocabulary logits. This layer has far fewer parameters than the full model but its gradient directly reflects the training objective.

The output-layer gradient for a data point x is:

$$\nabla_{W_{\text{head}}} \ell(x, \theta) = \frac{\partial \ell}{\partial \text{logits}} \cdot h_x^\top$$

where h_x is the final hidden state and $\frac{\partial \ell}{\partial \text{logits}}$ is the softmax loss gradient. This requires only a single matrix multiplication and does not backpropagate through transformer layers.

Technical Formulation

Standard gradient-based selection scores candidate x as:

$$s(x) = \langle \nabla_{\theta} \ell(x, \theta), \nabla_{\theta} \mathcal{L}(\mathcal{V}, \theta) \rangle$$

LESSER replaces θ with the output-layer parameters θ_L :

$$s_{\text{LESSER}}(x) = \langle \nabla_{\theta_L} \ell(x, \theta), \nabla_{\theta_L} \mathcal{L}(\mathcal{V}, \theta) \rangle$$

The key empirical finding—that batch-level alignment is preserved—can be stated as follows. Let B_{full} and B_{LESSER} be the top- k batches selected by each method. Then empirically:

$$\langle \nabla_{\theta} \mathcal{L}(B_{\text{LESSER}}, \theta), \nabla_{\theta} \mathcal{L}(\mathcal{V}, \theta) \rangle \approx \langle \nabla_{\theta} \mathcal{L}(B_{\text{full}}, \theta), \nabla_{\theta} \mathcal{L}(\mathcal{V}, \theta) \rangle$$

The intuition is that the last layer aggregates information from all earlier layers, so its gradient captures the dominant component of the full gradient along the data manifold.

Learning or Inference Procedure

1. **Cache validation gradient:** Run a forward-backward pass on the validation set using only output-layer parameters. Cache $g_V = \nabla_{\theta_L} \mathcal{L}(\mathcal{V}, \theta)$.
2. **Score candidates:** For each candidate x_i , run a forward pass; compute $g_i = \nabla_{\theta_L} \ell(x_i, \theta)$ via one matrix multiplication; score $s_i = \langle g_i, g_V \rangle$.
3. **Select top- k :** Rank candidates by s_i ; select the top k .
4. **Post-train:** Fine-tune on selected data with standard SFT or RL objective.

What the Guarantee Says

There is no formal guarantee that output-layer ranking preserves full-gradient ranking at the sample level—and indeed it does not, in general. The operative guarantee is empirical and at the batch level: across all tested settings, batches selected by LESSER achieve gradient alignment with the validation set indistinguishable from those selected by full-gradient methods, and deliver matching downstream task performance.

Experimental Findings

- **FLOP reduction:** LESSER reduces feature-extraction cost by **9.7×** for SFT and **3.0×** for RL benchmarks.
- **Performance parity:** On downstream evaluations, LESSER matches full-gradient selection within measurement noise across all settings.
- **Sample vs. batch alignment:** Output-layer and full-gradient Spearman sample-ranking correlations vary widely, but batch-level gradient alignment with the validation set is consistently preserved.
- **Compatibility:** LESSER wraps LESS, DataInf, and other selection algorithms without modifying the selection logic.

Ablations and Interpretation

Layer depth sweep: Ablations over which layer’s gradients to use show a monotone improvement with depth; the output layer is best. Mid-layer gradients reduce cost but widen the quality gap.

Validation set size: Performance degrades gracefully as validation size shrinks; 128 examples suffice for reliable selection.

SFT vs. RL gap: The FLOP saving is larger for SFT (9.7×) than RL (3.0×) because RL requires gradients over longer generated sequences, making each full backward pass more expensive and the relative saving smaller.

Reference

Lyuxin David Zhang, Eric Wong, Surbhi Goel & Anton Xue. **LESSER: Post-Training Data Selection with Output-Layer Gradients**. arXiv:2610.03702, October 2026.

<https://arxiv.org/abs/2610.03702>

Keywords: video diffusion • physical consistency • seriality gap • autoregressive generation

S2PD: Serial-to-Parallel Diffusion for Physically and Logically Consistent Video Generation

A Two-Phase Diffusion Algorithm Resolves the Seriality Gap by Running Autoregressive Diffusion at High Noise and Parallel Diffusion at Low Noise

By Jeffrey Hu, Daniel Olmeda Reino & Ayush Tewari (Massachusetts Institute of Technology)

arXiv:2610.06847 • October 2026

Bidirectional video diffusion models violate physical laws and logical rules even when trained on unlimited in-distribution data—a structural obstacle called the *seriality gap*. S2PD resolves it with a single algorithmic change: run the denoiser autoregressively at high noise, then switch to parallel denoising at low noise, combining physical consistency with practical sampling speed.

AT A GLANCE

Problem: Bidirectional diffusion cannot reliably coordinate causally dependent video events, even with unlimited training data.

Why it matters: Physics simulations, game environments, and embodied AI all require causally consistent video generation.

Method: S2PD: autoregressive diffusion for the high-noise phase; parallel diffusion for the low-noise phase.

Result: More reliable rule-following than bidirectional baselines; faster sampling than fully autoregressive methods.

Evidence: Games, physical simulations, and real video benchmarks.

The Big Picture

Video diffusion models have reached remarkable visual quality, but a persistent blind spot—the *seriality gap*—limits their utility for scientific simulation, game generation, and embodied AI. Even when trained to convergence on procedurally generated data with perfect ground-truth labels, bidirectional diffusion models continue to violate physical laws: collision outcomes are wrong, logical sequential rules are broken, and causal chains are severed.

Prior work (arXiv:2607.13031) identified and characterized this gap. S2PD is the first method to close it without

sacrificing the sampling speed advantage of diffusion over fully autoregressive approaches.

Why Existing Approaches Fall Short

More denoising steps do not help. The seriality gap is not a capacity issue. For deterministic video prediction, additional denoising steps do not add effective serial computation—each step still processes all frames in parallel.

Architectural width does not help. Wider networks also fail to close the gap, confirming that the bottleneck is the absence of serial computation, not model capacity.

Fully autoregressive generation is slow. Frame-by-frame autoregressive generation provides the necessary serial compute but multiplies sampling time by the number of frames, making it impractical for high-resolution or long video generation.

Core Mechanism

The key insight is that different noise levels require different amounts of serial computation:

- **High noise** ($\sigma > \sigma^*$): the video is essentially pure noise; the denoiser must decide the *global causal structure*—which events happen, in what order, with what consequences. This requires serial computation to propagate causal dependencies frame by frame.
- **Low noise** ($\sigma \leq \sigma^*$): the global structure is already established; the denoiser only needs to refine visual detail, which can be done in parallel without violating consistency.

S2PD switches between autoregressive and parallel denoising at threshold σ^* .

Technical Formulation

Let $\mathbf{x}_\sigma$ be the noisy video at noise level σ , with frames $(x_{\sigma,1}, \dots, x_{\sigma,T})$. The S2PD denoising trajectory is:

Serial phase ($\sigma > \sigma^*$): denoise frame by frame,

$$\hat{x}_{0,t} = \epsilon_\theta(x_{\sigma,t}, \sigma \mid x_{0,1:t-1}), \quad t = 1, \dots, T$$

where conditioning on previously denoised frames is implemented via masked self-attention or cross-attention.

Parallel phase ($\sigma \leq \sigma^*$): denoise all frames jointly,

$$(\hat{x}_{0,1}, \dots, \hat{x}_{0,T}) = \epsilon_\theta(\mathbf{x}_\sigma, \sigma)$$

starting from the latent at the end of the serial phase.

Total sampling time scales as approximately:

$$\tau_{\text{S2PD}} \approx (T - 1) \tau_{\text{serial-step}} + \tau_{\text{parallel}}$$

substantially less than $T \tau_{\text{serial-step}}$ for fully autoregressive generation.

Learning or Inference Procedure

S2PD is a **training-free** inference algorithm applied to any pretrained video diffusion model with autoregressive conditioning support.

1. Sample Gaussian noise $\mathbf{x}_{\sigma_{\max}}$.
2. Run the DDIM solver serially (frame by frame) from $\sigma_{\max}$ to σ^* .
3. Run the solver in parallel over all frames from σ^* to 0.
4. Return the denoised video $\hat{\mathbf{x}}_0$.

The switching threshold σ^* is the only hyperparameter and is set to the midpoint of the noise schedule in all experiments.

What the Guarantee Says

No formal consistency guarantee is provided. The empirical claim is: S2PD produces fewer physical and logical violations than matched bidirectional baselines across all tested environments, while requiring less sampling time than fully autoregressive methods. The intuition is that the high-noise serial phase establishes the causal skeleton, making the low-noise parallel phase sufficient for detail refinement.

Experimental Findings

- **Games:** S2PD follows Tetris collision rules and Sokoban push logic more reliably than bidirectional baselines.
- **Physical simulations:** Lower rule-violation rates on billiards and rigid-body interaction benchmarks.
- **Real video:** Improved temporal stability on real-world video prediction while maintaining visual quality.
- **Sampling efficiency:** Substantially faster than fully autoregressive generation while closing the consistency gap.

Ablations and Interpretation

Switching threshold: Earlier switching (less serial computation) degrades consistency; later switching improves consistency but reduces efficiency gains. The midpoint is Pareto-optimal.

Phase ablations: Removing the parallel phase (fully serial) matches S2PD’s consistency but increases sampling time by $T\times$; removing the serial phase (fully parallel) restores the seriality gap.

Error analysis: Violations concentrate in frames requiring coordination with distant prior frames, confirming that the gap is a long-range causal dependency problem

rather than a local temporal one.

Reference

Jeffrey Hu, Daniel Olmeda Reino & Ayush Tewari. **S2PD: Serial-to-Parallel Diffusion for Physically and Logically Consistent Video Generation**. arXiv:2610.06847, October 2026.
<https://arxiv.org/abs/2610.06847>